MICROSERVICES COURSE

Session 5

Resilience Advanced

Bulkhead · TimeLimiter · Complete Defense Layer

Dr. ElSayed Mohamed Elsayed Baladoh

Tech Lead · Ph.D. · sayedbaladoh@yahoo.com

Wednesday

3:00 PM – 5:30 PM

Online Session

2.5 Hours

Phase 1

Session 5 of 29

WHERE WE ARE

Phase 1 — Foundation & Core Patterns

S1	S2	S3	S4	S5	S6	S7	S8
Architecture	Gateway Core	Gateway Adv.	Resilience CB	Resilience BH	OpenFeign	Saga+Kafka	Caching+Clinic
				● LIVE			

You are here — Session 5 of 8 in Phase 1. Last session: stop calling broken services. Today: survive SLOW ones too.

PROBLEM FIRST

Circuit Breaker Stops Broken Calls. But What About Slow Ones?

✅ Session 4 Solved

Payment Service is BROKEN (throws errors) → Circuit Breaker opens, fails fast, fallback returns PENDING.

❓ Still Unsolved

Payment Service is SLOW (10s instead of 200ms) but technically UP — Circuit Breaker doesn't see this as a failure yet.

- → 50 threads call Payment, each waits 10 seconds
- → Order Service thread pool: 200 threads total
- → After ~6 minutes: ALL 200 threads consumed
- → New orders cannot be created — even though Payment is "UP"

This is what Bulkhead and TimeLimiter solve today.

CORE CONCEPT — THE METAPHOR

Bulkhead — Watertight Compartments

“A ship's hull is divided into watertight compartments. If one floods, the others are sealed off — the ship stays afloat. One hole does not sink everything.”

Order→Product

✓ Unaffected, still fast

Order→Inventory

✓ Unaffected, still fast

Order→Payment

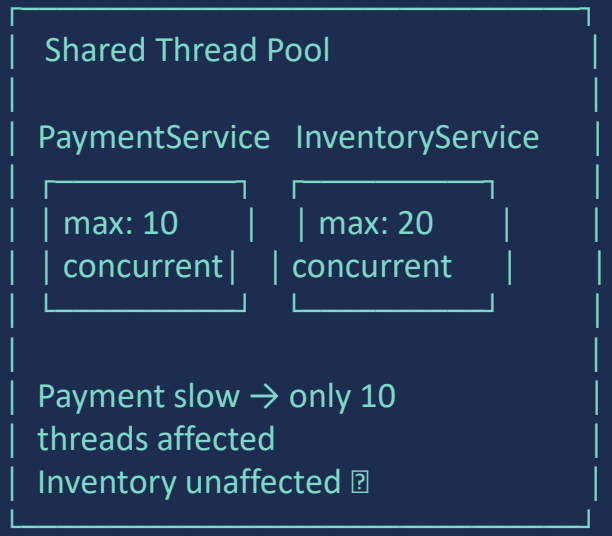
 Flooded — isolated here

We solved Circuit Breaker — stop calling a broken service. Bulkhead limits the BLAST RADIUS when a service is merely slow.

Without Bulkhead vs With Bulkhead

❌ WITHOUT Bulkhead

✅ WITH Bulkhead (Semaphore)



CORE CONCEPT — THE EFFECT

Without Bulkhead vs With Bulkhead

Without Bulkhead

- Payment Service becomes slow
- All Order Service threads pile up waiting
- Thread pool (200) fully exhausted
- Inventory & Product calls ALSO fail
- → Entire Order Service goes down

With Bulkhead

- Payment Service becomes slow
- Only 10 threads (the Bulkhead limit) are affected
- Remaining 190 threads stay free
- Inventory & Product calls work normally
- → Only Payment-related orders are QUEUED

ENGINEERING DECISION

Semaphore Bulkhead vs ThreadPool Bulkhead

	Semaphore Bulkhead	ThreadPool Bulkhead
How it works	Limits number of concurrent calls	Executes calls in separate thread pool
Caller thread	Blocks if limit reached	Returns immediately — non-blocking
Best for	Reactive / WebFlux services	Traditional blocking services
Config key	max-concurrent-calls	maxThreadPoolSize + queueCapacity
Our choice	**Used for Order→Payment**	Discussed as alternative

LIVE CODING

Configuring & Applying @Bulkhead

Goal

Add a Bulkhead that limits Order Service to max 10 concurrent calls to Payment Service — isolating any slowness

application.yml — Order Service

```
resilience4j:
  bulkhead:
    instances:
      paymentService:
        max-concurrent-calls: 10    # max 10 simultaneous calls to Payment
        max-wait-duration: 0ms      # do NOT wait , do NOT queue — fail immediately if full
        # max-wait-duration: 500ms  # alternative: wait up to 500ms for a slot
```

LIVE CODING

Configuring & Applying @Bulkhead

OrderService.java - Apply @Bulkhead annotation

```
// OrderService.java — add @Bulkhead to existing method

@Bulkhead(name = "paymentService", fallbackMethod = "bulkheadFallback")
@CircuitBreaker(name = "paymentService", fallbackMethod = "paymentFallback")
@Retry(name = "paymentService")
public OrderResponse createOrder(OrderRequest request) {
    PaymentResponse payment = paymentService.processPayment(
        new PaymentRequest(request.getAmount())
    );
    return new OrderResponse("CONFIRMED", payment.getTransactionId());
}

// Bulkhead fallback — called when concurrent limit exceeded
public OrderResponse bulkheadFallback(OrderRequest request, BulkheadFullException ex) {
    log.warn("[BULKHEAD] Concurrent limit reached: {}", ex.getMessage());
    return new OrderResponse("QUEUED", "System busy — your order is queued");
}
```

DEMO

Testing the Bulkhead — 15 Concurrent Requests

Send 15 requests simultaneously (background jobs)

```
for i in {1..15}; do curl -X POST .../api/orders ... & done; wait
```

10 requests

CONFIRMED or CB/Retry fallback

reached Payment Service

5 requests

QUEUED — "System busy"

never reached Payment Service

The Bulkhead protected Payment Service from overload — 5 requests never even reached it.

PROBLEM FIRST — PART 2

Why "Slow" Is More Dangerous Than "Down"

1. Payment Service is responding... but taking 30 seconds per request
2. Order Service has a 60-second HTTP timeout — so it waits
3. 10 requests × 30 seconds = 300 seconds of thread blocking
4. Thread pool has 200 threads. All consumed after ~6 minutes.
5. New orders cannot be created — but Payment is still technically UP

Solution: Do not wait indefinitely. If Payment takes > 2 seconds → fail fast. TimeLimiter enforces this at the Resilience4j level — cleaner than HTTP timeouts.

CORE CONCEPT

TimeLimiter — Cut the Wait at 2 Seconds

Without TimeLimiter:

```
Waiting... waiting... waiting... (30 seconds, thread blocked the whole time)
```

With TimeLimiter (timeout-duration: 2s):

```
0-2s: wait
```

```
2s: TIMEOUT → cancel future → timeoutFallback() → PENDING returned immediately
```

HTTP timeout is a blunt instrument — it just cuts the connection. TimeLimiter is smarter: it cancels the Future AND records the failure with the CircuitBreaker.

ENGINEERING DECISION

Why Does TimeLimiter Require CompletableFuture?

Synchronous Method

```
public OrderResponse createOrder(...)
```

Java cannot reliably interrupt a running thread mid-execution. There is no "handle" to cancel — the thread just keeps blocking.

CompletableFuture<T>

```
public CompletableFuture<OrderResponse>
createOrderAsync(...)
```

A Future IS a cancellable handle. TimeLimiter calls future.cancel() at the 2-second mark — this is the only way to enforce a hard timeout in Java.

This is why we wrap the synchronous payment call in CompletableFuture.supplyAsync() next.

LIVE CODING · STEP 1-2

Configuring TimeLimiter

application.yml — Order Service

```
resilience4j:
  timelimiter:
    instances:
      paymentService:
        timeout-duration: 2s          # fail if Payment takes > 2 seconds
        cancel-running-future: true   # cancel the thread on timeout
```

LIVE CODING · STEP 1-2

Simulating Slowness

application.yml — Payment Service

```
payment:
  failure-rate: 0.5          # 50% failure — change to 0.0 to test recovery (Session 4)
  delay-ms: 3000             # 3000 (3s) to trigger the 2s TimeLimiter on Order Service.
```

PaymentController.java — simulate slowness

```
// PaymentController.java — add configurable delay
@Value("${payment.delay-ms:0}")
private long delayMs;

@PostMapping
public ResponseEntity<PaymentResponse> processPayment(@RequestBody PaymentRequest request)
    throws InterruptedException {
    if (delayMs > 0) {
        Thread.sleep(delayMs);  // simulate slow service
    }
    // ... rest of existing logic
}
```

LIVE CODING · STEP 3

@TimeLimiter with Async Wrapper

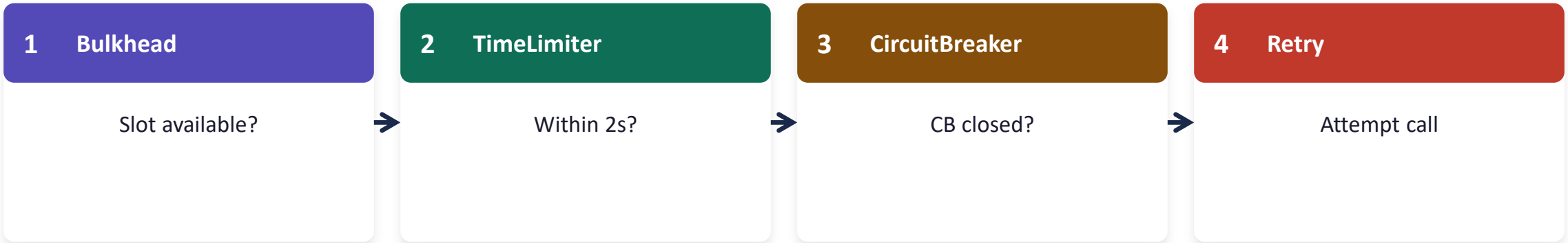
OrderService.java

```
@Bulkhead(name = "paymentService", fallbackMethod = "bulkheadFallback")
@TimeLimiter(name = "paymentService", fallbackMethod = "timeoutFallback")
@CircuitBreaker(name = "paymentService", fallbackMethod = "paymentFallback")
@Retry(name = "paymentService")
public CompletableFuture<OrderResponse> createOrderAsync(OrderRequest request) {
    return CompletableFuture.supplyAsync(() -> {
        PaymentResponse payment = paymentService.processPayment(
            new PaymentRequest(request.getAmount()));
        return new OrderResponse("CONFIRMED", payment.getTransactionId());
    });
}

// TimeLimiter fallback — called on TimeoutException
public CompletableFuture<OrderResponse> timeoutFallback(
        OrderRequest request, TimeoutException ex) {
    log.warn("[TIMEOUT] Payment exceeded 2s limit: {}", ex.getMessage());
    return CompletableFuture.completedFuture(
        new OrderResponse("PENDING", "Payment timed out"));
}
```

PAUSE & CONNECT THE DOTS

Four Annotations — What Order Do They Run In?



Outermost protects first. If Bulkhead's slot is full, the call never even reaches TimeLimiter, CircuitBreaker, or Retry.

Next: we update the controller to handle the async response — the last piece of plumbing.

LIVE CODING · STEP 4

Updating the Controller for Async

OrderController.java

```
@PostMapping
public CompletableFuture<ResponseEntity<OrderResponse>> createOrder(
        @RequestBody OrderRequest request) {
    return orderService.createOrderAsync(request)
        .thenApply(response -> ResponseEntity.ok(response));
}

// Spring MVC handles CompletableFuture transparently
// Client sees a normal synchronous HTTP response — async is internal only
```

Test: Set payment.delay-ms=3000. Call POST /api/orders. See PENDING in ~2s, not 3s. CB records the timeout as a failure.

DEMO

Testing the Timeout — 2 Seconds, Not 3

Set payment.delay-ms=3000 in Payment Service config

```
curl -X POST http://localhost:8080/api/orders -d '{"amount":100}'
```

Timeline:

```
t=0s                    Request sent
t=2s                    [TIMEOUT] Payment exceeded 2s limit
t=2s                    Response: {"status":"PENDING", ...}
t=3s (never reached)    Payment Service would have responded here
```

✅ **PENDING returned at the 2-second mark — not 3. TimeLimiter cut the wait exactly as configured.**

The Complete Defense Layer — Full Resilience Stack

✅ Normal Flow

```
Client → Gateway
   ↓
Order Service
   ↓  (Bulkhead: max 10 concurrent)
Payment Service
   ↓
APPROVED
   ↓
Order: CONFIRMED
```

❌ Failure Flow

```
Client → Gateway
   ↓
Order Service
   ↓  (Bulkhead full → BulkheadFullException)
TimeLimiter → timeout after 2s
Retry → 3 attempts
CircuitBreaker → OPEN
   ↓
Fallback: Order PENDING
```

The Complete Defense Layer — Execution Order

1. Bulkhead	Is there a concurrent slot available?	*BulkheadFullException → bulkheadFallback()*
2. TimeLimiter	Did the call complete within 2 seconds?	*TimeoutException → timeoutFallback()*
3. CircuitBreaker	Is the circuit OPEN (too many recent failures)?	*Rejected → paymentFallback()*
4. Retry	Did the call fail? Retry up to 3 times w/ backoff.	*Each retry is a new attempt through the CB*
5. Actual Call	paymentService.processPayment() finally runs	

SUMMARY — PATTERN TO FAILURE MODE

ENGINEERING DECISION

Which Pattern Solves Which Problem?

Pattern	Failure Mode Addressed	What You Observe
Bulkhead	Thread pool exhaustion	QUEUED response when >10 concurrent calls
TimeLimiter	Slow service (not down)	PENDING after 2s — not waiting 30s
CircuitBreaker	Repeated failures	OPEN state — fast fail, service rests
Retry	Transient single failure	Retry #1,2,3 in logs with backoff
Fallback	Unrecoverable failure	Graceful PENDING — user not left with error

HANDS-ON

Lab 3B — Full Resilience Stack on Order→Payment

1	**Add Semaphore Bulkhead**	max-concurrent-calls: 10, bulkheadFallback returns QUEUED
2	**Add TimeLimiter with Async Wrapper**	2s timeout, CompletableFuture, timeoutFallback returns PENDING
3	**Write Unit Tests**	Minimum 3 tests covering Bulkhead + TimeLimiter behaviour

DEFINITION OF DONE

Lab 3B Acceptance Criteria

- [x] 15 concurrent requests: ~10 CONFIRMED/fallback, ~5 QUEUED
- [x] payment.delay-ms=3000 → PENDING response arrives in ~2 seconds
- [x] Actuator shows both CB state AND bulkhead metrics
- [x] Logs show [BULKHEAD] and [TIMEOUT] prefixes for fallbacks
- [x] Full stack on createOrderAsync(): @Bulkhead+@TimeLimiter+@CircuitBreaker+@Retry
- [x] All 3 unit tests pass: mvn test
- [x] Commit: session-05: add-bulkhead-and-timelimiter-to-order-payment

PLATFORM PROGRESS

What We Added to the Platform Today

Capability Added

Complete Resilience Layer

Full defense against all failure modes: thread exhaustion, slow services, repeated failures, transient errors.

Order Service — Full Stack

- @Bulkhead: max 10 concurrent → QUEUED
- @TimeLimiter: 2s timeout → PENDING
- @CircuitBreaker + @Retry (from Session 4)
- Fallback: graceful PENDING response

- Sessions 1-3: Config + Eureka + Product + Gateway (secured)
- Session 4: Payment + Order — Circuit Breaker + Retry
- Session 5: Complete Resilience Layer — Bulkhead + TimeLimiter

Config	Eureka	Product	Gateway Sec.	Payment	Order Resil.	Inventory	Notif.

Daily Quiz

8 Questions · 10 Minutes

Topics: Bulkhead Types · TimeLimiter · CompletableFuture · Full Stack Execution Order

BEFORE SESSION 6

Checkpoint Commit & Homework

Checkpoint — Definition of Done

- ☐ 15 concurrent → ~5 QUEUED
- ☐ 3s delay → PENDING in ~2s
- ☐ Both fallbacks return correct status
- ☐ Full annotation stack on createOrderAsync()
- ☐ Pushed: session-05: ...

Pre-Session 6 Reading (15 min)

OpenFeign & Declarative HTTP Clients
spring.io/projects/spring-cloud-openfeign

Knowledge Check Questions for Session 6:

- What is the advantage of OpenFeign over RestTemplate?
- When should Order Service call Inventory synchronously?
- What does @EnableFeignClients do?

TROUBLESHOOTING

Common Issues & Solutions

TimeLimiter not triggering on slow payment	Verify cancel-running-future: true and method returns CompletableFuture — not synchronous
timeoutFallback() not being called	Fallback return type must be CompletableFuture<OrderResponse> — must match exactly
Bulkhead never triggers (always passes through)	max-wait-duration: 0ms required — default waits indefinitely, same as no Bulkhead
BulkheadFullException not caught by CB	Bulkhead and CB have separate fallbacks — does not propagate to paymentFallback()
Concurrent test not triggering Bulkhead	HTTP client may serialize requests — use background & jobs or k6/Gatling for true concurrency

Session 5 Complete

Resilience Layer Finished — Bulkhead + TimeLimiter + CB + Retry

Next: Session 6 — Inter-Service Communication: OpenFeign & WebClient · Monday, 3:00–5:30 PM, Online